Convolutional Neural Networks

CS3410 Deep Learning - Lecture 7

In this lecture we cover convolution, padding, stride, pooling and how gradients flow backwards through a convolutional layer.

Why not fully connected layers

A fully connected layer on a 224 by 224 colour image needs over 150000 weights for a single unit, which is wasteful and overfits quickly.
Images have local structure: nearby pixels are related, and a useful feature detector is useful wherever it appears in the image.
Convolution exploits both properties through local connectivity and weight sharing, so the same kernel is applied at every position.

The convolution operation

A convolutional layer slides a small kernel over the input and computes a weighted sum at each position. The kernel weights are learned.
For a two dimensional input the discrete convolution is written as:

```
y[i,j] = sum_m sum_n x[i+m, j+n] * w[m,n] + b
```

Padding

Padding adds a border of extra values, usually zeros, around the input before the kernel is applied.

Padding is necessary for two reasons. First, without padding the output is smaller than the input, so a deep stack of convolutions shrinks the feature map until no spatial information remains. Second, pixels at the edge of the image are covered by fewer kernel positions than pixels in the centre, so edge information is under represented.

Same padding keeps the output the same size as the input. Valid padding applies no padding at all and the output is smaller.

Stride

Stride is the step size the kernel moves between applications.
A stride of one moves the kernel by a single position and produces the densest output. A stride of two halves the spatial resolution and is a cheap alternative to pooling.
The output size for input size W, kernel size K, padding P and stride S is given by the following expression:

```
W_out = floor((W - K + 2P) / S) + 1
```

Pooling

Pooling reduces the spatial size of a feature map by summarising small neighbourhoods. Max pooling takes the largest value in each window and average pooling takes the mean.
Pooling gives a small amount of translation invariance, because moving the input by one pixel often does not change which value is the maximum.
Max pooling has no learned parameters, so it adds no weights.

Backpropagation through convolution

The gradient with respect to the kernel weights is itself a convolution between the input and the gradient arriving from the next layer. Because weights are shared across all positions, the gradient for one weight is the sum of contributions from every position where it was used. The gradient with respect to the input is a convolution of the incoming gradient with the kernel rotated by 180 degrees, often called a transposed convolution.

```
dL/dw[m,n] = sum_i sum_j dL/dy[i,j] * x[i+m, j+n]
```

Receptive field

The receptive field of a unit is the region of the original input that can influence its value.
Stacking convolutions grows the receptive field linearly, while adding stride or pooling grows it multiplicatively. This is why deep networks with small three by three kernels can still see large structures.

Summary

Convolution gives local connectivity and weight sharing.

Padding preserves spatial size and treats edge pixels fairly.

Stride controls how far the kernel moves and reduces resolution.

Pooling summarises neighbourhoods and adds translation invariance.

Gradients flow back through convolution as another convolution.